llmctl docs/dependency-layout

Dependency layout: one source of truth

Field	Value
Revision	2
Created	2026-10-21
Last modified	2026-10-21
Status	active

llmctl does **not** vendor `constitution`, `submodules/containers` or `submodules/superspec` (owner mandate 2026-10-21: reduce redundancy, one source of truth, always the latest, main submodules live in `<root>/submodules` or `<root>/constitution/submodules`, never nested chains). llmctl itself lives inside the constitution hub at `constitution/submodules/llmctl`; a nested `constitution` here created a hub -> llmctl -> hub cycle. The hub now vendors `containers` and `superspec` under `<hub>/submodules/<name>` (owner decision 2026-10-21), so llmctl resolves them instead of pinning private copies. llmctl's own real dependencies stay ordinary submodules: `submodules/llama.cpp` and `submodules/colibri` (`git submodule update --init`).

Hub-engine placement (diagram)

Source: `diagrams/hub-engine-placement.mmd` (rendered svg). llmctl is a hub engine (Constitution §11.4.28(C), §11.4.31): consumed by reference, depth 1, never nesting the canon.

```
flowchart TB
  subgraph HUB["constitution hub (Constitution 11.4.28(C), 11.4.31)"]
    CON["constitution (canon, CLAUDE.md, Constitution.md)"]
    subgraph ENG["constitution/submodules (hub engines, depth 1, recursive_init false)"]
      LLM["llmctl (this repository)"]
      CT["claude_toolkit (provider aliases, JEV decision layer)"]
      OD["open_design (reference checkout)"]
    end
    CNT["submodules/containers (vendored once by the hub)"]
    SS["submodules/superspec (third party)"]
  end
  LLM -- "lib/resolve_deps.sh: env, then parent hub, then root, then siblings" --> CON
  LLM -- "resolved, never nested" --> CNT
  LLM -- "resolved, never nested" --> SS
  LLM -. "own third-party pins only" .-> LC["submodules/llama.cpp and submodules/colibri"
  CT -- "calls llmctl decide / endpoint" --> LLM
  GATE["CM-HUB-ENGINE-NO-OWNORG-NESTING: FAIL on a nested own-org repo"] -. checks .-> EN
  MAN["helix-deps.yaml in every own-org repo (CM-HELIX-DEPS-MANIFEST-PRESENT)"] -. descri
```

How a dependency is found (`lib/resolve_deps.sh`)

```
bash lib/resolve_deps.sh constitution   # or containers / superspec /
        --all
```

First match wins: 1. env override `LLMCTL_CONSTITUTION_DIR` / `LLMCTL_CONTAINERS_DIR` / `LLMCTL_SUPERSPEC_DIR` (a relative value is made absolute; an empty or invalid override is an error, never a silent fall-through); 2. the parent hub, when llmctl is at `<hub>/submodules/llmctl` (constitution = `<hub>`, containers/superspec = `<hub>/submodules/<name>`, vendored by the hub); 3. `<root>/constitution`, `<root>/submodules/<name>` (a consumer's own copy, Constitution section 11.4.76); 4. sibling

clones for standalone use; 5. ancestor directories (`A/constitution` , `A/submodules/<name>`); 6. host fallback for the constitution only, **opt-in**: used only when `LLMCTL_HOST_FALLBACK_DIR` is set (there is no built-in host path). Nothing resolved: exit 1 with an actionable message. A real `<root>/constitution` directory (leftover of the old gitlink) is used but warned about.

Fixed repo-relative paths: `scripts/link_deps.sh`

`go.mod` (`replace digital.vasic.containers => ./submodules/containers`) and the carriers' `@constitution/CLAUDE.md` import need fixed paths. `bash scripts/link_deps.sh` creates the three **untracked, .gitignored** symlinks (`constitution` , `submodules/containers` , `submodules/superspec`) to whatever the resolver found (`--check` verifies only; `link_deps.sh containers` links just that one). `make deps` / `make test` run it, and `llmctl setup` / `llmctl build decide|all` link `containers` themselves (and stop with the resolver's message if it cannot be found). A real directory in the way is never replaced. Inside the hub the links are relative (`constitution` -> `../..` , `submodules/<name>` -> `../../<name>`), so the `@constitution/CLAUDE.md` import keeps working with no extra clone; elsewhere they are absolute.

Symlink cycle inside the hub. `<hub>/submodules/llmctl/constitution` -> `<hub>` makes the hub contain itself. Tools that follow symlinks (`find -L` , `grep -R` , indexers, archivers) will loop or double-count: exclude `submodules/llmctl/constitution` , `submodules/llmctl/submodules/containers` and `submodules/llmctl/submodules/superspec` from code indexes and scans (the links are gitignored and never archived; do not edit canon for this).

Outside the hub on a bare clone: clone the constitution next to llmctl, or export the `LLMCTL_*_DIR` variables, then run `make deps` . The release preflight only verifies llmctl's own pins (llama.cpp, colibri) and prints the commits of the resolved dependencies so the release evidence records them.

Upgrading an existing checkout (leftovers)

git does not delete a populated submodule worktree when its gitlink is removed. After pulling, an old checkout keeps REAL directories `constitution/`, `submodules/containers/`, `submodules/superspec/` (gitignored, so hidden). `scripts/link_deps.sh` reports each as `LEFTOVER <path>` and exits 1 (so `make deps` fails) until you clean it up **by hand** - no llmctl code moves, renames or deletes a directory of yours:

- a shared copy exists (hub / `LLMCTL_*_DIR`): verify nothing is lost - each of `git -C <path> status --porcelain`, `git -C <path> log --branches --not --remotes --oneline` and `git -C <path> stash list` prints nothing - then `rm -rf <path>`;
- the leftover is the only copy found: `git clone <path> <a directory OUTSIDE this repository>` and `export LLMCTL_<DEP>_DIR=<that directory>`; leave `<path>` untouched until the clone is verified. **Never move or rename it to a name inside this repository**: it would stay an untracked embedded repository (`git add -A` stages it as a gitlink) and its `core.worktree` would still point at the old path, so git commands inside it would act on the shared checkout (`.gitignore` also ignores renamed siblings of the three paths, as defence in depth only);
- then re-run `scripts/link_deps.sh`.

Until then the resolver warns (every dependency, also when the hub copy wins) and `llmctl setup` / `llmctl build decide|all` build from a leftover `containers` only if it is the resolver's own answer and, as a git checkout, clean; otherwise they stop with the LEFTOVER message. Go-building test suites call `require_containers_or_skip` (`tests/helpers.sh`, opt-in) and SKIP loudly when containers is unresolvable.